

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

Национальный исследовательский ядерный университет «МИФИ»



**Институт интеллектуальных кибернетических систем
КАФЕДРА КИБЕРНЕТИКИ (№ 22)**

Требования к ПО

Б20-524

Корначук М.В.

Лукичев А.Н.

Кузнецов М.М.

Латыпов И.В.

Москва 2023

Содержание

1. Введение

1.1 Назначение

Данная Спецификация Требований к ПО создана для описания функциональных и нефункциональных требований к системе отслеживания статусов кабинетов в НИЯУ МИФИ. Этот документ будет служить основой для дальнейшей разработки, предоставляя описание необходимых функций и характеристик. Спецификация также предназначена для обеспечения чёткости требований в процессе взаимодействия с заказчиком и разработчиками, а также предоставления актуальной информации о системе на данный момент.

1.2 Область действия

Система для отслеживания статусов кабинетов представляет собой сложное программное обеспечение, предназначенное для упрощения процесса управления и использования учебных и лабораторных кабинетов в НИЯУ МИФИ. Система позволяет пользователям получать актуальную информацию о доступности кабинетов, их оснащении и расписании занятий, а также осуществлять бронирование кабинетов. Более подробная информация представлена в следующих разделах.

1.3 Определения, акронимы, сокращения

JSON (JavaScript Object Notation) - Легкий формат обмена данными, удобный для чтения и записи человеком и для анализа и генерации машиной. Используется для передачи данных между веб-приложением и сервером.

API (Application Programming Interface) - Набор определений и протоколов для построения и интеграции программного обеспечения. API позволяет различным приложениям взаимодействовать друг с другом.

Токен - Строка, используемая в качестве ключа доступа к API. В Telegram ботах, токен предоставляется при создании бота и используется для аутентификации запросов к API Telegram.

HTTP (HyperText Transfer Protocol) - Это прикладной протокол для передачи гипертекстовых документов, таких как HTML. Он используется для передачи данных между веб-сервером и браузером, что позволяет пользователям просматривать веб-страницы. HTTP является основой данных общения в World Wide Web. Протокол определяет методы и форматы сообщений, используемые для запроса и передачи данных. В его основе лежит концепция

запроса (от клиента) и ответа (от сервера), и он работает поверх TCP/IP протокола. HTTP позволяет клиентам отправлять запросы на сервер для загрузки веб-страниц, изображений, файлов и других данных, а серверам - отвечать на эти запросы, отправляя запрошенные ресурсы.

HTTPS (HyperText Transfer Protocol Secure) - Расширение HTTP протокола, поддерживающее шифрование для повышения безопасности данных. Используется для защищенного обмена данными между клиентом и сервером.

HTTP-запрос - Запрос, отправляемый по протоколу HTTP для получения или отправки данных между клиентом и сервером, особенно в контексте взаимодействия с Telegram API.

POST запрос - Метод запроса HTTP протокола для отправки данных на сервер, например, при отправке формы или создании/обновлении данных в API.

GET запрос - Метод запроса HTTP, используемый для получения данных с сервера, например, при запросе обновлений от серверов Telegram.

БД (База Данных) - Организованная коллекция данных, хранящаяся и доступная для обработки с помощью компьютерных систем. Базы данных используются для хранения, поиска и обновления данных.

chat_id - Уникальный идентификатор чата в Telegram, используемый для определения места отправки сообщения.

1.4 Ссылки

1. DO178B - Стандарт для сертификации программного обеспечения в авиационных системах [rtca_do178b].
2. Официальная документация Telegram API [telegram2023botsapi].

1.5 Обзор

Спецификация требований разделена на несколько основных частей: общее описание системы, включая сравнение с аналогами, описание её функционала и потенциальных пользователей, а также ограничения и предположения, влияющие на разработку. В последующих разделах детально описаны требования к системе, включая требования к внешним интерфейсам, атрибутам ПО, производительности и другим важным аспектам.

2. Общее описание

2.1 Перспективы изделия

Система для отслеживания статусов кабинетов НИЯУ МИФИ представляет собой интегрированное программное решение, предназначенное для автоматизации и упрощения процессов управления и использования учебных и лабораторных помещений в университете. Это веб-приложение предоставляет возможность поиска свободных кабинетов, а также получения актуальной информации о расписании и оснащении кабинетов.

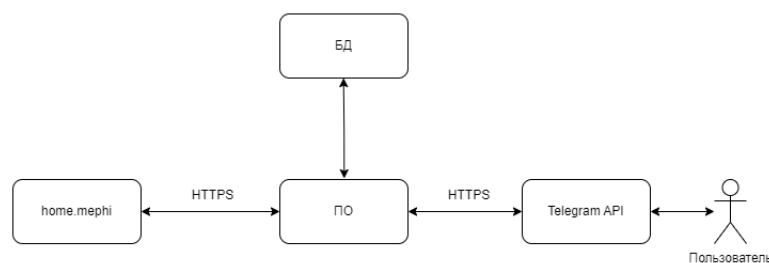


Рисунок 2.1 – Архитектура системы

- **Пользователь** - Конечные пользователи системы, которые взаимодействуют с системой через Telegram бота.
- **Telegram API** - Платформа, через которую пользовательские запросы передаются в ПО системы. Обеспечивает коммуникацию между пользователем и системой.
- **Программное Обеспечение (ПО)** - Основной компонент системы, обрабатывающий данные и взаимодействующий с базой данных и внешними API.
- **База Данных (БД)** - Хранилище данных, содержащее информацию о кабинетах, расписаниях и прочих данных, необходимых для работы системы.
- **home.mephi** - Внешний сервис университета, предоставляющий данные, необходимые для интеграции с системой.

Вся коммуникация между ПО и Telegram API, а также между ПО и home.mephi осуществляется по протоколу HTTPS, который обеспечивает безопасное шифрование передаваемых данных.

Дополнительно, система предусматривает возможность масштабирования и добавления новых функций, что делает ее адаптивной к изменяющимся требованиям университета.

2.2 Функции изделия

1. **Отображение карты МИФИ:** Показывает расположение кабинетов в университете.
2. **Поиск свободных кабинетов:** Позволяет пользователям находить доступные кабинеты на определенную дату и время.
3. **Информация о кабинетах:** Предоставляет данные о оснащении кабинетов, а также о расписании занятий.
4. **Интеграция с Telegram:** Обеспечивает взаимодействие с пользователями через Telegram бота, включая отправку уведомлений и команд.
5. **Логирование и отчетность:** Ведет журналы работы системы и предоставляет отчеты для администраторов.

2.3 Характеристики пользователя

Категории пользователей системы:

1. **Студенты и преподаватели:** Основные пользователи системы, использующие ее для поиска кабинетов.
2. **Административный персонал:** Управляет системой, собирает обратную связь
3. **Разработчики и техническая поддержка:** Отвечают за обслуживание системы, ее обновление и устранение неполадок.

2.4 Ограничения

- **Limitation.Server.ProgrammingLanguage:** Для реализации ПО использовать язык программирования Go с версией 1.20.0 или совместимой с ней.
- **Limitation.Server.ListenerType:** Обмен данными внутри приложения и при обращении к внешним API должен происходить по протоколу не HTTP/1.1 или новее.
- **Limitation.Server.HomeMephiAPI:** ПО должно обращаться к portalу home.mephi не чаще 20 раз в минуту, реализуя механизм контроля запросов для соблюдения данного ограничения. Должна быть реализована система очереди запросов с возможностью равномерного распределения запросов во времени для предотвращения превышения лимита. (Например, если лимит запросов в минуту - 20, нужно отправлять не более 20 запросов в минуту и для равномерного распределения запросов, отправлять запрос раз в 3 секунды)
- **Limitation.Server.TelegramAPI:** ПО должно обращаться к API Telegram не чаще, чем 30 сообщений в секунду.

3. Конкретные требования

3.1 Требования к внешним интерфейсам - Interface:

3.1.1 Аппаратные интерфейсы - .Hardware

- **Interface.Hardware.Processors:** Сервер должен использовать архитектуру процессора x86-64 или ARM.
- **Interface.Hardware.RAM:** Сервер должен работать без сбоев при наличии 4 GB оперативной памяти.
- **Interface.Hardware.Networking:** Сервер должен работать без сбоев при использовании любого сетевого интерфейса, обеспечивающего минимальную стабильную скорость 512 KB/S.

3.1.2 Интерфейсы ПО - .Software

- **Interface.Software.OS:** Сервер должен быть совместим с ОС Windows 10 и старше и дистрибутивами с ядром Linux версии 4.2 и старше.
- **Interface.Software.OS.Files:** Сервер должен иметь возможность создавать, удалять, модифицировать и читать файлы.
- **Interface.Software.OS.Networking:** Сервер должен иметь возможность взаимодействовать с внешней сетью по протоколу TCP/IP.
- **Interface.Network.Access:** Сервер должен иметь возможность обращаться по протоколу HTTPS к API home.merphi и API Telegram и они должны быть доступны.
- **Interface.Software.DataBase:** Сервер должен иметь возможность устанавливать соединение с СУБД PostgreSQL и взаимодействовать с ней посредством SQL запросов.
- **Interface.Software.APIRequestTelegram:** ПО должно отправлять запрос к Telegram API для получения сообщений от пользователей следующим образом: HTTP-запрос: GET <https://api.telegram.org/bot{token}/getUpdates> Параметры: {token}: Строка, содержащая токен авторизации бота, предоставленный Telegram при регистрации бота. Данный запрос должен отправляться не реже чем раз в 2 секунды. В дальнейшем в документе, понятие "получить через запрос" будет означать получить через запросом, способом описанным выше
- **Interface.Software.APISendMessageTelegram:** При отправке ответа пользователю, ПО

должно отправлять запрос к Telegram API следующим образом:

HTTP-запрос: POST `https://api.telegram.org/bot{token}/sendMessage`

Тело запроса (JSON):

```
{
  "chat_id": "{chat_id}",
  "text": "{text_message}"
}
```

Параметры: {chat_id}: Целочисленный идентификатор чата, в который должно быть отправлено сообщение. {text_message}: Текст сообщения, отправляемого пользователю. В дальнейшем в документе, понятие "отправить сообщение", "вывести сообщение" будет означать отправить запрос, способом описанным выше.

- **Interface.Software.APICreateCustomMenuTelegram:** ПО должно быть способно создавать произвольное меню с кнопками в Telegram-боте. Для этого должен быть реализован следующий функционал:

HTTP-запрос:

Метод: POST

URL: `https://api.telegram.org/bot{token}/sendMessage`

Тело запроса (в формате JSON): { "chat_id": "{chat_id}", "text": "{menu_text}", "reply_markup": { "keyboard": [[{ "text": "{button_1}" }], [{ "text": "{button_2}" }], ... [{ "text": "{button_n}" }],], "resize_keyboard": true, "one_time_keyboard": true } }

Параметры: {token}: Строка, содержащая токен авторизации бота, предоставленный Telegram.

{chat_id}: Целочисленный идентификатор чата с пользователем.

{menu_text}: Текст, отображаемый в сообщении меню.

{n}: Количество кнопок

{button_1}, {button_2}, ..., {button_n}: Текст на каждой из n кнопок соответственно.

При выборе опции из главного меню, ПО получит сообщение с текстом опции. В дальнейшем документе под созданием меню будем подразумевать запрос выше с соответствующими параметрами.

- **Interface.Software.APIGetRoomsScheduleHomeMephi:** ПО должно получать информацию (синхронизировать) с портала home.mephi по следующему протоколу: HTTP-запрос: GET `https://home.mephi.ru/api/v1/schedule/rooms`.

3.1.3 Интерфейсы связи

- **Interface.Connection.DataCommunicationProtocol:** Обмен данными внутри приложения и при обращении к внешним API должен происходить по протоколу не HTTP/1.1 или новее. SQL запросов.

3.2 Функциональные требования - Functional:

3.2.1 Сервис логики бота - .BotLogic

- **Functional.BotLogic.StartCommandResponse:** При получении (см. Interface.Software.APIRequestTelegram) команды /start от пользователя, ПО должно отправлять ему в ответ (см. Interface.Software.APISendMessageTelegram) приветственное сообщение, содержащее список команд, описанный в Functional.BotLogic.HelpCommandResponse. Пользователю должно отображаться главное меню, описанное в (см. Component.BotLogic.CreateCustomMenu).
- **Functional.BotLogic.HelpCommandResponse:** При получении (см. Interface.Software.APIRequestTelegram) команды /help от пользователя, ПО должно отправлять ему в ответ (см. Interface.Software.APISendMessageTelegram) приветственное сообщение, содержащее список команд. ПО должно получать список команд из БД, список должен содержать {название команды} - {описание команды}.
- **Functional.BotLogic.MainMenuOptions:** В главном меню, ПО должно предоставлять пользователю опции меню (см. Interface.Software.APICreateCustomMenuTelegram): 'Поиск свободного кабинета' и 'Информация о кабинете'.
- **Functional.BotLogic.OptionSelectionScenario:** При выборе опции из главного меню, ПО должно переводить пользователя в соответствующий сценарий: поиск кабинета или получение информации о кабинете.
- **Functional.Search.FreeRoomSearch:** В сценарии 'Поиск свободного кабинета', ПО должно показывать изображение карты МИФИ, полученную из централизованной БД, и позволять выбрать корпус из списка корпусов, полученных из централизованной БД.
- **Functional.Search.BuildingSelection:** После выбора корпуса, ПО должно запросить у пользователя дату и предоставить опции для выбора: 'Сегодня', 'Завтра' или ввод конкретной даты. При выборе 'Ввод конкретной даты', ПО должно отображать подсказку о формате ввода и предоставлять возможность ввода путём отправки сообщения.